

Introduction to GPU programming

GPU Programming – M2 DiPaC, Université Paris-Saclay – Session 1

Oguz Kaya

Assistant Professor
Université Paris-Saclay and LISN, Gif-sur-Yvette, France

Objectives

- Some reminders on parallel computing, applications, and architecture.
- Getting to know the architecture of a GPU (vs. CPU)
- Rapid overview to GPU architecture and programming
- Understanding the execution model of a CUDA program
- Using multiple blocks and threads in a CUDA kernel
- Learning the basic syntax for a CUDA program
- Mastering data allocation and transfer between CPU and GPU
- Compilation, execution, and launching mechanisms of GPU programs
- Elaborating these concepts with an example (array multiplication)

Outline

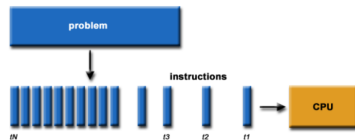
- 1 Introduction
- 2 Why parallel computing?
- 3 Parallel architecture
- 4 First look at GPU programming
- 5 Execution of a CUDA program
- 6 CUDA syntax, memory allocation and data transfer
- 7 Example: multiplying an array by blocks and threads
- 8 Compilation and execution

- 1 Introduction
- 2 Why parallel computing?
- 3 Parallel architecture
- 4 First look at GPU programming
- 5 Execution of a CUDA program
- 6 CUDA syntax, memory allocation and data transfer
- 7 Example: multiplying an array by blocks and threads
- 8 Compilation and execution

Sequential program

Traditionally, algorithms and software are based on a **sequential execution**:

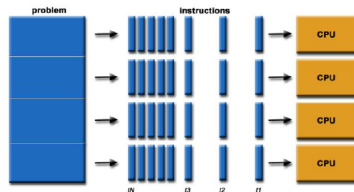
- A problem is split into instructions.
- These instructions are executed **sequentially** one after another.
- They are executed by **only one processor**
- At a given time step, at most one instruction is executed.
- The performance is determined mostly by the **frequency** (Hz) of the processor.



Parallel programming

Parallel programming enables the use of multiple processing units to solve a given problem

- A problem is split into independent parts that can be treated simultaneously.
- Each part is split into instructions.
- Instructions of different parts are executed **in parallel** by using multiple processors.
- The performance is determined by the
 - processor frequency
 - number of processors
 - degree of parallelization of the problem.

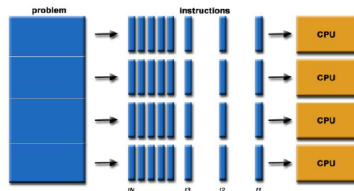


- 1 Introduction
- 2 Why parallel computing?**
- 3 Parallel architecture
- 4 First look at GPU programming
- 5 Execution of a CUDA program
- 6 CUDA syntax, memory allocation and data transfer
- 7 Example: multiplying an array by blocks and threads
- 8 Compilation and execution

Applications of parallel computing

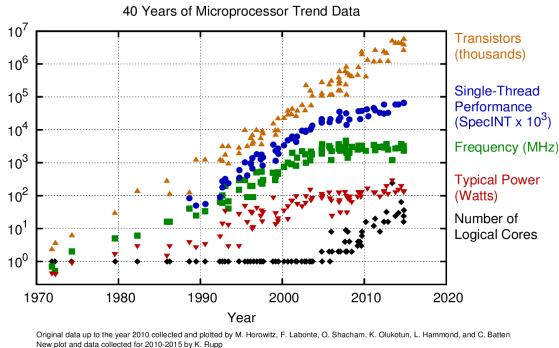
Numerous applications requiring a lot of compute power in diverse domains:

- Scientific computing: Simulations for physics, chemistry, biology, climate, quantum computing, etc.
- Neural network training and inference
- Computer graphics: Rendering, video games, animations, 3D modeling, etc.
- Operating systems: Linux, Windows, Android, etc.
- and many others



Why parallel? Because the clock stopped

- 2002: Pentium 4 Northwood, **3.06 GHz**.
2024: Core i9-14900KS, **6.2 GHz**. Two decades, $2\times$.
- Transistor counts kept doubling (Moore's law) but **Dennard scaling ended** around 2005: power $\propto CV^2f$, and V could no longer shrink, so f hit the **power wall**
- Single-thread performance has been nearly flat since
- The transistors went into **more cores**, **wider vector units**, and **accelerators** – GPUs above all
- Consequence: a program that does not run in parallel does not get faster on new hardware



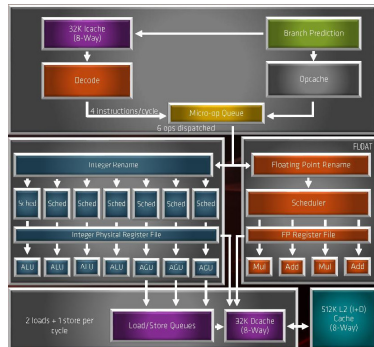
40 years of microprocessor trend data. K. Rupp, after Horowitz et al.

- 1 Introduction
- 2 Why parallel computing?
- 3 Parallel architecture**
- 4 First look at GPU programming
- 5 Execution of a CUDA program
- 6 CUDA syntax, memory allocation and data transfer
- 7 Example: multiplying an array by blocks and threads
- 8 Compilation and execution

CPU

“Central Processing Unit”, or CPU, generally consists of

- Multiple processing units (cores)
- Multiple memory levels (registers, L1, L2, L3, RAM)
- Multiple execution ports in each core (ALUs, vector units)
- Vector units for SIMD parallelism (AVX2, AVX512, Arm Neon, ...)
- Simultaneous execution of multiple threads (2-4)
- Capable of exploiting the instruction level parallelism (ILP) with micro-op buffer, instruction reordering, register renaming, etc.
- A good portion of hardware is dedicated to managing ILP + cache



The AMD Zen 2 architecture

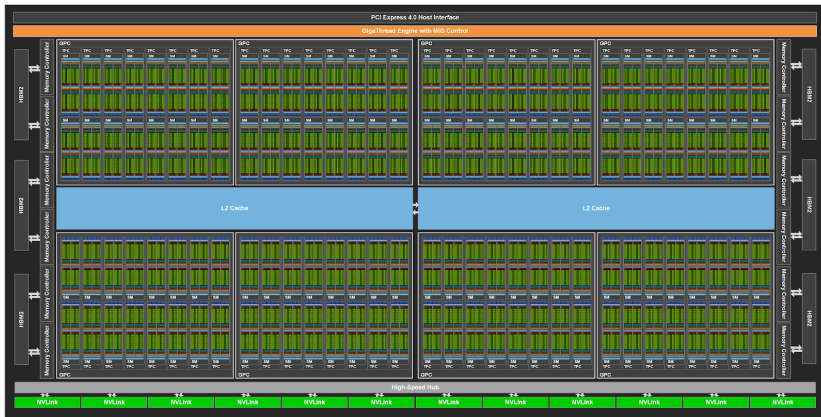
“Graphical Processing Unit”, or GPU, is a vector processing unit consisting of

- Multiple execution units (streaming multiprocessors - SMs)
- Multiple memory levels (registers, shared memory, L1, L2, RAM)
- Multiple vector units (1-8) that are wide (16-32 floats) in each SM.
- Simultaneous execution of **thousands** of threads
- Giant **register file** (64K 32-bit registers per SM)
- Very fast context switching among threads, since threads reside in SMs simultaneously
- Most of the hardware is dedicated to vector compute units, very high performance potential.
- Parallelization requires some effort



The Nvidia Ampere architecture

GPU (cont.)

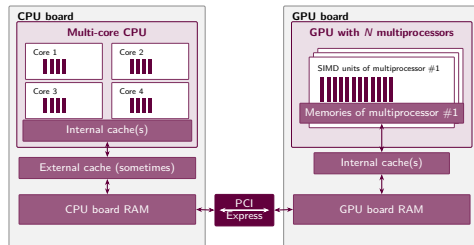


Nvidia Ampere architecture

CPU-GPU Overview

Overview of a CPU with a GPU

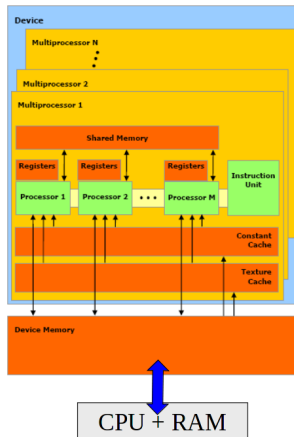
- The CPU uses the GPU as a coprocessor for **data-parallel** computations: the same operation over many elements.
- Both the CPU and GPU are **multi-core** and **vector processors** with a specific memory hierarchy.
- Data transfer occurs over the PCI Express bus (64 GB/s in each direction for PCIe 5.0 x16).
- Each has its own RAM; by default, neither accesses the other's directly.
- This is evolving: on Grace Hopper (CPU+GPU on one module, NVLink-C2C) the two memories form one coherent address space.



Zoom on GPU architecture

A GPU is a collection of N independent SIMD processors sharing a global memory

- N streaming “multiprocessors” (SM)
- Each SM is a SIMD processor having
 - k synchronized processors ($k = 32$), in other words, **CUDA cores**
 - 1 shared instruction decoder
 - 3 types of memories shared among all k processors
 - 64K registers per SM, distributed among the threads (up to 255 per **thread**)
 - To fully utilize each SM, you need to launch **at least 32 threads (per block)**
 - To fully utilize the **whole GPU**, you need **at least one block per SM** (in practice several, so the scheduler has work in reserve)



Some numbers for the capacity of GPU architectures

Nvidia data-centre GPUs, one per generation (SXM parts, full configuration¹)

	K40 2013	M40 2015	P100 2016	V100 2017	A100 2020	H100 2022	B200 2024
SMs	15	24	56	80	108	132	160
FP32 cores / SM	192	128	64	64	64	128	128
FP32 cores / GPU	2880	3072	3584	5120	6912	16896	20480
FP64 cores / SM	64	4	32	32	32	64	64
FP64 cores / GPU	960	96	1792	2560	3456	8448	10240
Tensor cores / SM	—	—	—	8	4	4	4
Tensor cores / GPU	—	—	—	640	432	528	640
Peak FP32 (TFLOPS)	5.0	6.8	10.6	15.7	19.5	67	75
Peak FP64 (TFLOPS)	1.7	0.2	5.3	7.8	9.7	34	40
Peak Tensor FP16 (TFLOPS)	—	—	—	125	312	989	2250

Some compute capabilities do not evolve monotonically

¹ Shipping SKUs may have a few SMs disabled (e.g. B200: 148 of 160). Tensor peak is dense FP16; sparsity doubles it. Sources: Nvidia whitepapers.

Supercomputer / cluster

An ensemble of connected machines (CPU+GPU) over a network

- Connection with a network having a topology (ring, grid, torus, clique, etc.)
- Optimized communication libraries adapted to the topology
- Capable of addressing very large-scale problems
- Today we are well past the **exaflop** scale (10^{18} floating point operations per second, or flops/s)
 - #1 **LineShine** (Shenzhen, China): **2.20 exaflops/s**
 - #2 **El Capitan** (LLNL, USA): 1.81 exaflops/s
 - #4 **JUPITER Booster** (Jülich, Germany): 1.0 exaflops/s, Europe's first exascale machine
 - 5 exascale systems in total; the whole TOP500 adds up to 18.7 exaflops/s (June 2026 list)
 - In France: **Jean Zay** (IDRIS, Orsay): 125.9 petaflops/s; 1456 H100, 416 A100, 1832 V100



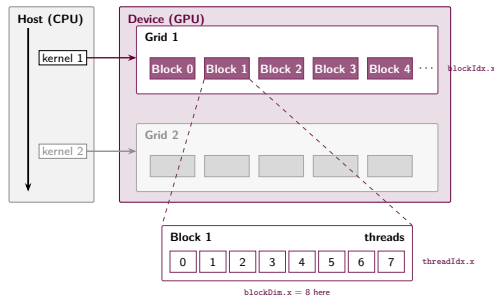
Joliot-Curie supercomputer,
300K CPU cores, 1024 GPUs

- 1 Introduction
- 2 Why parallel computing?
- 3 Parallel architecture
- 4 First look at GPU programming**
- 5 Execution of a CUDA program
- 6 CUDA syntax, memory allocation and data transfer
- 7 Example: multiplying an array by blocks and threads
- 8 Compilation and execution

GPU programming

GPU programming follows the **SIMT** model: *single instruction, multiple threads*.

- SIMD (CPU vector units): one instruction applied to one wide register.
SIMT: one instruction executed by *many threads* at once, each with its own registers and its own data – as if each vector lane were a thread.
- The GPU has many identical **streaming multiprocessors** (SMs), each with many **CUDA cores**; a core executes one thread at a time.
- A **kernel** is a function that runs on the GPU; every thread executes it.
- A **block** is a group of threads executed concurrently on the same SM.
- The blocks of one launch form a **grid**. A block tells itself apart from the others by its index, `blockIdx.x`.



CUDA programming

CUDA, the programming language developed by Nvidia and based on C/C++, consists of

- A compiler (nvcc)
- A runtime library (`cuda_runtime.h`: memory, transfers, launches, errors; nvcc includes it automatically in every .cu file)
- And many other libraries with optimized kernels (cuBLAS, cuDNN, cuSOLVER, cuTENSOR, RAPIDS, etc.)
- CUDA is **Nvidia-only**; portable alternatives exist:
 - **HIP/ROCm** (AMD, close to CUDA), **OpenCL**, **SYCL**/oneAPI
 - Directive-based: **OpenMP** target offload, **OpenACC**
 - Higher level: **Kokkos**, **Triton**, array/AI frameworks
- We use CUDA: it exposes the hardware most directly



Nvidia Ampere RTX 3090 GPU

Hello World in OpenMP

- **omp.h** is the OpenMP library header that provides necessary functions (e.g., to obtain `thid`, `numth`, etc.)
- **#pragma omp parallel** creates 3 threads that execute **the same code** and **simultaneously** and **asynchronously**
- Each thread has a unique identifier between 0 and 2 (or $P - 1$ if we create P threads)
- Possible outputs?
 - 3! possibilities due to asynchronous execution

```
#include <stdio>
#include "omp.h"

int main(void)
{
    #pragma omp parallel num_threads(3)
    {
        int thid = omp_get_thread_num();
        int numth = omp_get_num_threads();
        printf("Hello from thread %d/%d.\n", thid, numth);
    }
    return 0;
}
```

Hello World in CUDA

- **cuda_runtime.h** provides the necessary functions (nvcc includes it for you).
- **__global__** marks a GPU kernel (a function is CPU-side by default)
- Inside a kernel, **blockIdx.x** gives the block identifier and **gridDim.x** the number of blocks
- **<<< 3, 1 >>>** creates 3 blocks (1 thread each) that execute **the same code asynchronously**
 - Each block has an identifier between 0 and 2 (0 to $P - 1$ for P blocks)
- The launch is **asynchronous**: the CPU carries on. **cudaDeviceSynchronize()** blocks until the GPU is done, like an **OpenMP barrier**
 - Without it, main could return before anything is printed
- Possible outputs? 3! orderings, because execution is asynchronous

```
#include <stdio>
#include <cuda_runtime.h>

__global__ void helloWorld()
{
    printf("Hello from block %d/%d\n",
        blockIdx.x, gridDim.x);
}

int main(void)
{
    helloWorld<<<3, 1>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

Multiply an array in OpenMP

Multiply each element of an array **A[N]** by a scalar **c**.

- Is this program correct?
 - No! Each element is multiplied 4 times!

```
#include <stdio>
#include "omp.h"

#define N 1024

int main(void)
{
    float A[N];
    float c = 2.0f;
    // Initialization
    for (int i = 0; i < N; i++) { A[i] = i; }
    #pragma omp parallel num_threads(4)
    {
        for (int i = 0; i < N; i++) {
            A[i] *= c;
        }
    }
    return 0;
}
```

Multiply an array in OpenMP (cont.)

Multiply each element of an array **A[N]** by a scalar **c**.

- Each thread still executes the same code
- This time, the execution is differentiated by **thid**
- Using P threads, each thread iterates over N/P consecutive elements of **A[N]**
- Be careful for the last thread if N is not divisible by P .

```
#include <stdio>
#include "omp.h"

#define N 1024

int main(void)
{
    float A[N];
    float c = 2.0f;
    // Initialization
    for (int i = 0; i < N; i++) { A[i] = i; }
    #pragma omp parallel num_threads(4)
    {
        int thid = omp_get_thread_num();
        int numth = omp_get_num_threads();
        int elemsPerThread = N / numth;
        int begin = thid * elemsPerThread;
        int end;
        if (thid < numth - 1) { // All but the last thread
            end = (thid + 1) * elemsPerThread;
        } else { // The last thread
            end = N;
        }
        for (int i = begin; i < end; i++) {
            A[i] *= c;
        }
    }
    return 0;
}
```

Multiply an array in CUDA

Multiply each element of an array **A[N]** by a scalar **c**.

- **__device__** declares an array in GPU memory; **__global__** a function that runs on the GPU
- Data must be copied to the GPU before, and back after, with **cudaMemcpy...** (§7)
- Each block executes the same code; **blockIdx.x** tells them apart
- With P blocks, each block handles N/P consecutive elements – careful with the last one if $P \nmid N$
- This program is **perfectly correct...** and **terrible for a GPU**: 4 threads on a chip built for tens of thousands. Fixed in §8.

```
#include <stdio>
#include <cuda_runtime.h>

#define N 1024
float A[N];
float c = 2.0f;

__device__ float dA[N];

__global__ void multiplyArray(int n, float c)
{
    int elemsPerBlock = n / gridDim.x;
    int begin = blockIdx.x * elemsPerBlock;
    int end;
    if (blockIdx.x < gridDim.x - 1) {
        end = (blockIdx.x + 1) * elemsPerBlock;
    } else {
        end = n;
    }
    for (int i = begin; i < end; i++) { dA[i] *= c; }
}

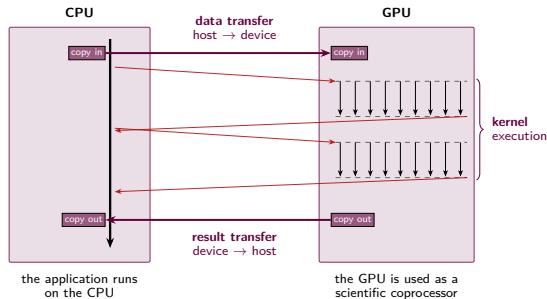
int main(void)
{
    // Initialization
    for (int i = 0; i < N; i++) { A[i] = i; }
    // Copy the array to the GPU
    cudaMemcpyToSymbol(dA, A, N * sizeof(float), 0,
        cudaMemcpyHostToDevice);
    multiplyArray<<<4, 1>>>>(N, c);
    // Copy the multiplied array back to the CPU
    cudaMemcpyFromSymbol(A, dA, N * sizeof(float), 0,
        cudaMemcpyDeviceToHost);
    printf("%f\n", A[2]);
    return 0;
}
```

- 1 Introduction
- 2 Why parallel computing?
- 3 Parallel architecture
- 4 First look at GPU programming
- 5 Execution of a CUDA program**
- 6 CUDA syntax, memory allocation and data transfer
- 7 Example: multiplying an array by blocks and threads
- 8 Compilation and execution

Execution principle

The program mainly runs on the CPU with calls to GPU functions.

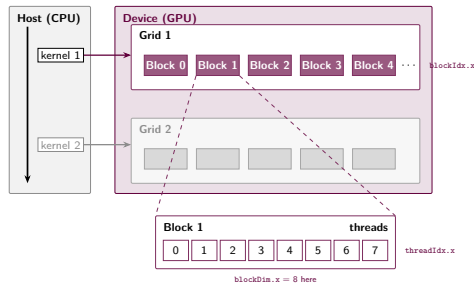
- Before/after launching a kernel, data must be transferred
- Data transfers should be minimized for efficiency
- Each kernel call is non-blocking (i.e., the CPU continues execution), but it can be made blocking if desired



Execution of a grid of thread blocks

The CPU launches a kernel; the GPU runs it with a set of threads.

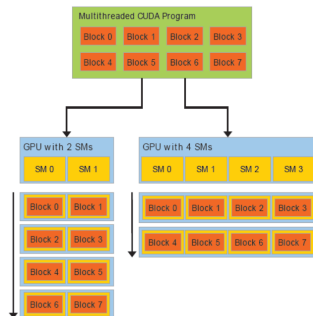
- All threads are identical: they execute the same code
- Threads are grouped into **blocks** (1 to 1024 threads; in practice a multiple of 32)
- Each block runs on one SM; the blocks of a launch form a **grid**, spread over all SMs
- Launch enough blocks and threads to cover the whole iteration domain of the problem
- Threads of the **same block** can share resources and synchronise. **Threads of different blocks cannot.**



Execution of a grid of thread blocks (cont.)

The CPU launches the execution of a kernel with a set of GPU threads.

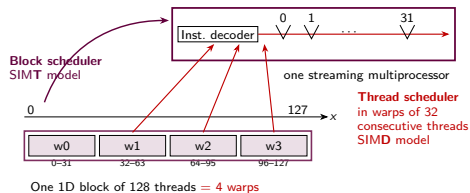
- The block scheduler distributes the blocks across the different SMs with dynamic scheduling.
- GPUs with different architectures can execute the same grid of thread blocks without any problem (with a distribution specific to their architecture, managed by the scheduler).



Warps: the unit of execution

Make blocks out of a whole number of **warps**

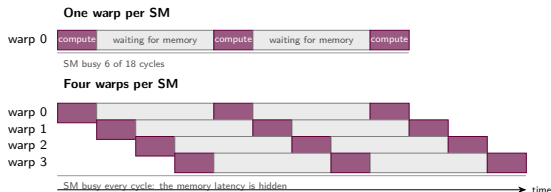
- One instruction decoder drives 32 CUDA cores, i.e. 32 threads at once
- Each group of 32 consecutive threads of a block is a **warp**: the unit the hardware actually schedules
- The SM's scheduler picks a warp of a resident block and issues its next instruction to the 32 cores
- A block of 100 threads is 4 warps, the last one 3/4 empty – hence “a multiple of 32”



Hiding memory latency: keep many warps resident

A global-memory access takes hundreds of cycles. The SM does not wait: it switches warps.

- All resident warps keep their registers in the SM, so switching between them costs **nothing**
- While one warp waits for memory, the scheduler issues instructions from another
- With enough warps in flight, the memory latency is **hidden** and the SM stays busy
- So: many **light threads**, many **blocks**
– little work per thread, and let the hardware interleave them



The choice of the number of blocks and threads

How many blocks/grid and threads/block should I use?

- The thread scheduler wants **many warps in reserve**, to hide memory latency (previous slide)
- The block scheduler wants **many not-too-large blocks**, so that
 - every SM gets work, with blocks in reserve
 - several blocks fit on one SM at a time (limited by registers and shared memory) and their memory accesses overlap
- In general, **128–256 threads per block** works well (min 1, max 1024).
- Optimal choice by experimentation, or with Nvidia's tools: **Nsight Compute** (occupancy analysis) and `cudaOccupancyMaxPotentialBlockSize()`

From thread to array element: the global index

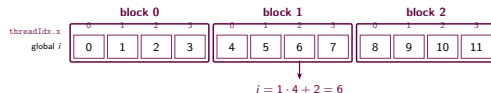
Every thread runs the same kernel. What makes it work on *its* element? Its **global index**:

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$$

- `threadIdx.x`: the thread's rank in its block,
 $0 \dots \text{blockDim.x} - 1$
- `blockIdx.x`: the block's rank in the grid,
 $0 \dots \text{gridDim.x} - 1$
- Launch enough threads to cover N elements:
 $\lceil N/\text{tpb} \rceil$ blocks, i.e. round *up*

```
int tpb = 256;  
int nb = (N + tpb - 1) / tpb;  
kernel<<<nb, tpb>>>(N, ...);
```

- The last block may overshoot N : guard with
if ($i < N$) in the kernel



- 1 Introduction
- 2 Why parallel computing?
- 3 Parallel architecture
- 4 First look at GPU programming
- 5 Execution of a CUDA program
- 6 CUDA syntax, memory allocation and data transfer**
- 7 Example: multiplying an array by blocks and threads
- 8 Compilation and execution

CUDA Qualifiers

A qualifier is a keyword that differentiates CPU/GPU functions and variables in a CUDA program.

	<code>--device--</code>	<code>--host--</code> (default)	<code>--global--</code>
Functions	Called on GPU Runs on GPU	Called on CPU Runs on CPU	Called on CPU Runs on GPU
Variables	<code>--device--</code> GPU global memory Lifetime of the application Accessible from GPU and CPU code	<code>--constant--</code> GPU constant memory Lifetime of the application Written by CPU code, read by GPU code	<code>--shared--</code> Shared memory of one SM Lifetime of the thread block Accessible from GPU code; used to cache GPU global memory

Qualifiers distinguish the GPU and CPU parts of the code.

Before anything else: check for errors

- Every runtime call returns a `cudaError_t`; `cudaSuccess` means fine, `cudaGetErrorString()` explains anything else
- Ignore it, and a failed `cudaMalloc` only shows up as garbage much later
- A **kernel launch returns nothing**; its errors are caught at two moments:
 - the launch itself (bad grid, no device): `cudaGetLastError()` right after
 - the kernel while running (out-of-bounds access): reported by the *next* synchronising call, e.g. `cudaDeviceSynchronize()`
- Wrap it in a macro once, use it on every call – as the listings do from here on

```
#ifndef CUDA_CHECK_H
#define CUDA_CHECK_H
#include <stdio>
#include <stdlib>
#include <cuda_runtime.h>

// Wrap every CUDA runtime call: on failure, print
// file:line and the message, then abort.
#define CUDA_CHECK(call) do { \
    cudaError_t err = (call); \
    if (err != cudaSuccess) { \
        fprintf(stderr, "CUDA error at %s:%d: %s\n", \
            __FILE__, __LINE__, cudaGetErrorString(err)); \
        exit(EXIT_FAILURE); \
    } } while (0)

// A kernel launch returns nothing; check it in two steps:
//   cudaGetLastError()    -> errors of the launch itself
//   cudaDeviceSynchronize() -> errors raised while running
#define CUDA_CHECK_KERNEL() do { \
    CUDA_CHECK(cudaGetLastError()); \
    CUDA_CHECK(cudaDeviceSynchronize()); } while (0)

#endif
```

Allocation of an array on GPU

- The prefix `__device__` differentiates the declaration of a static GPU array from a CPU array.
- The static GPU array must be declared outside of functions (as global variables)
- Dynamic arrays on GPU are allocated using the `cudaMalloc` function.

```
#include <cstdlib>
#include "cuda_check.h"

#define N 1024

// Static global array on the CPU
float arrCPU[N];

// Static global array on the GPU
__device__ float arrGPU[N];

int main(void)
{
    // Dynamic array on the CPU
    float *dynCPU = (float *) malloc(N * sizeof(float));

    // Dynamic array on the GPU
    float *dynGPU;
    CUDA_CHECK(cudaMalloc((void **) &dynGPU, N * sizeof(float)));

    CUDA_CHECK(cudaFree(dynGPU));
    free(dynCPU);
    return 0;
}
```

Copying a static array between a CPU and GPU

```
#include "cuda_check.h"

#define N 1024

// Static array on the CPU
float arrCPU[N];

// Static array on the GPU
__device__ float arrGPU[N];

int main(void)
{
    // Copy a static CPU array onto a static GPU array
    CUDA_CHECK(cudaMemcpyToSymbol(arrGPU, arrCPU,
        sizeof(float) * N, 0, cudaMemcpyHostToDevice));

    // Copy a static GPU array onto a static CPU array
    CUDA_CHECK(cudaMemcpyFromSymbol(arrCPU, arrGPU,
        sizeof(float) * N, 0, cudaMemcpyDeviceToHost));

    return 0;
}
```

Copying a dynamic array between a CPU and GPU

```
#include <cstdlib>
#include "cuda_check.h"

#define N 1024

int main(void)
{
    // Dynamic array on the CPU
    float *arrCPU = (float *) malloc(N * sizeof(float));

    // Dynamic array on the GPU
    float *arrGPU;
    CUDA_CHECK(cudaMalloc((void **) &arrGPU, N * sizeof(float)));

    // Copy a dynamic CPU array onto a dynamic GPU array
    CUDA_CHECK(cudaMemcpy(arrGPU, arrCPU, sizeof(float) * N,
        cudaMemcpyHostToDevice));

    // Copy a dynamic GPU array onto a dynamic CPU array
    CUDA_CHECK(cudaMemcpy(arrCPU, arrGPU, sizeof(float) * N,
        cudaMemcpyDeviceToHost));

    CUDA_CHECK(cudaFree(arrGPU));
    free(arrCPU);
    return 0;
}
```

cudaMemcpy is synchronous: it returns only once the copy is done, and it waits for any kernel launched before it. That is why the examples that follow copy results back without an explicit `cudaDeviceSynchronize()`.

- 1 Introduction
- 2 Why parallel computing?
- 3 Parallel architecture
- 4 First look at GPU programming
- 5 Execution of a CUDA program
- 6 CUDA syntax, memory allocation and data transfer
- 7 Example: multiplying an array by blocks and threads**
- 8 Compilation and execution

Multiply an array by blocks in CUDA (improved)

Multiply each element of an array $A[N]$ by a scalar c .

- `__global__` defines the function on the GPU.
 - This allows using `blockIdx.x`
- Each block always executes the same code.
- Each block performs 1 operation, so N blocks must be launched to cover the entire array/computation domain.
- Execution is differentiated by `blockIdx.x`.

```
#include <stdio>
#include <cuda_runtime.h>

#define N 1024
float A[N];
float c = 2.0f;

__device__ float dA[N];

__global__ void multiplyArray(int n, float c)
{
    int i = blockIdx.x;
    dA[i] *= c;
}

int main(void)
{
    // Initialization
    for (int i = 0; i < N; i++) { A[i] = i; }
    // Copy the array to the GPU
    cudaMemcpyToSymbol(dA, A, N * sizeof(float), 0,
        cudaMemcpyHostToDevice);
    multiplyArray<<<N, 1>>>(N, c);
    // Copy the multiplied array back to the CPU
    cudaMemcpyFromSymbol(A, dA, N * sizeof(float), 0,
        cudaMemcpyDeviceToHost);
    printf("%f\n", A[2]);
    return 0;
}
```

Multiply an array by blocks and threads in CUDA

Multiply each element of an array **A[N]** by a scalar **c**.

- **__global__** defines the function on the GPU.
 - This allows using **blockIdx.x**, **blockDim.x**, and **threadIdx.x**
- Each thread and block always executes the same code.
- We use **blockSize** threads per block.
- Each thread performs 1 operation, so **N / blockSize** blocks must be launched to cover the entire array/computation domain.
- Execution is differentiated by **blockIdx.x** and **threadIdx.x**.
- Be careful of array overflow (if **N** is not divisible by **blockSize**)

```
#include <stdio>
#include <cuda_runtime.h>

#define N 1024
float A[N];
float c = 2.0f;

__device__ float dA[N];

__global__ void multiplyArray(int n, float c)
{
    int i = threadIdx.x + blockIdx.x * blockDim.x;
    if (i < n)
        dA[i] *= c;
}

int main(void)
{
    // Initialization
    for (int i = 0; i < N; i++) { A[i] = i; }
    // Copy the array to the GPU
    cudaMemcpyToSymbol(dA, A, N * sizeof(float), 0,
        cudaMemcpyHostToDevice);
    int blockSize = 128;
    int numBlocks = N / blockSize;
    if (N % blockSize) numBlocks++;
    multiplyArray<<<numBlocks, blockSize>>>(N, c);
    // Copy the multiplied array back to the CPU
    cudaMemcpyFromSymbol(A, dA, N * sizeof(float), 0,
        cudaMemcpyDeviceToHost);
    printf("%f\n", A[2]);
    return 0;
}
```

Hello World, revisited: blocks *and* threads

- Same program as slide 18, now launched with **2 blocks of 4 threads**
- Every thread prints its two coordinates: `threadIdx.x` in the block, `blockIdx.x` in the grid
- 8 lines of output, in an order you cannot predict – and it may differ from one run to the next
- Question: which pairs (`blockIdx.x`, `threadIdx.x`) map to global index $i = 5$?
To $i = 0$?

```
#include <stdio>
#include <cuda_runtime.h>

__global__ void helloWorld()
{
    printf("Hello from threadIdx=%d/blockIdx=%d\n",
          threadIdx.x, blockIdx.x, gridDim.x);
}

int main(void)
{
    helloWorld<<<2, 4>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

- 1 Introduction
- 2 Why parallel computing?
- 3 Parallel architecture
- 4 First look at GPU programming
- 5 Execution of a CUDA program
- 6 CUDA syntax, memory allocation and data transfer
- 7 Example: multiplying an array by blocks and threads
- 8 Compilation and execution**

Compilation and execution of a CUDA program

- Source files must have the extension **.cu** (e.g., program.cu)
- **Compilation:** `nvcc program.cu -o program`
 - Target architecture: `-arch=native` compiles for the GPU in the machine; or name it, e.g. `-arch=sm_80` (Ampere), `sm_90` (Hopper), `sm_100` (Blackwell)
 - Find out what you have with `nvidia-smi`
 - `nvcc` drives a host C++ compiler (g++/clang) for the CPU part; on the lab machines, load the CUDA module first
- **Execution:** `./program`

Contact

Oguz Kaya

Université Paris-Saclay and LISN, Gif-sur-Yvette, France

oguz.kaya@universite-paris-saclay.fr

www.oguzkaya.com

Course material: github.com/kayaogz/2627-m2dipac-gpuprogramming